PWA PLAY STORE DEPLOYMENT GUIDE

Complete Step-by-Step Technical Documentation for Trusted Web Activity (TWA) Publishing

Document Reference	LAW-PWA-PLAYSTORE-v1.0	**Author**	Antigravity AI
Target Platform	Google Play Store (Android)	**Status**	Release-Ready

1. Introduction & Architecture Overview

Publishing a Progressive Web App (PWA) to the Google Play Store is achieved using **Trusted Web Activity (TWA)** protocol. Instead of rebuilding the application as a native Android client using Java/Kotlin, a lightweight native wrapper app (an APK or AAB file) is generated. This wrapper opens your fully functional HTTPS PWA inside a secure, full-screen Chrome custom tab browser instance, removing the browser address bar and giving it a native feel.

> 💡 **Key Advantage:** Any updates made to your PWA code (frontend bugs, CSS fixes, design updates) are deployed instantly to your users without requiring them to download a new version from the Play Store. You only need to republish the app bundle if you change native parameters like the app icon, splash screen, or app name.

2. Pre-requisites Checklist

Before initiating the build, make sure you have prepared the following elements:

- **Google Play Console Account:** A registered Developer Account ($25 one-time registration fee).
- **HTTPS PWA URL:** A fully secure web app URL (e.g. `https://lawoncall.in`) with an active SSL certificate (HTTP links are rejected).
- **High-Quality Icons & Assets:**
  - One `512x512 px` PNG app icon.
  - One `1024x500 px` PNG feature graphic.
  - A minimum of 2-4 device screenshots (both phone and tablet ratios).

- **Valid Manifest & Service Worker:** Your web application must pass all Lighthouse PWA validation criteria (offline loading support is mandatory).

3. Generating the Android App Bundle (AAB)

There are two primary methods for generating the deployment files: using the **Bubblewrap CLI** (local developer method) or **PWABuilder** (cloud GUI tool).

Method A: Using Bubblewrap CLI (Recommended for Developers)

Bubblewrap is Google's official command-line tool for packaging PWAs as TWA Android applications. It automates the generation of signing keys and app bundles.

1. Install the Bubblewrap CLI tool globally using NPM:

```
npm install -g @bubblewrap/cli
```

2. Initialize the project directory and fetch your live PWA's manifest JSON:

```
bubblewrap init --manifest=https://lawoncall.in/manifest.webmanifest
```

   Bubblewrap will download the manifest file, automatically extract your color codes, icon references, and basic parameters, and ask you to confirm them in the terminal.

3. Generate the release signing key (Keystore):

```
bubblewrap keygen
```

> ⚠️ **CRITICAL WARNING:** Keep the generated keystore file (usually `android.keystore`) and passwords safe! If you lose this key, you will not be able to push updates or new releases to the Play Store for this application.

4. Build your signed production Android App Bundle (AAB):

```
bubblewrap build
```

   This command compiles the wrapper code, signs it using the keystore, and generates the final file: `app-release-signed.aab` in your build folder.

Method B: Using PWABuilder (Easy GUI Tool)

If you prefer not to use the command line, you can package the app online:

1. Go to **pwabuilder.com** in your browser.

2. Enter your live PWA URL (e.g. `https://lawoncall.in`) and click **Start**.
3. Once PWA checks are completed, click **Package for Store** -> **Android** -> **Generate Package**.
4. Download the generated ZIP file, which contains:
   - `app-release.aab` (The Android App Bundle to upload to Play Console).
   - `signing.keystore` (Your private signing key).
   - `assetlinks.json` (The digital asset links file).

4. Configuring Digital Asset Links (Crucial Step)

To remove the browser address bar from the top of your wrapper app, you must prove ownership of the domain. This is done by hosting a digital handshake file at your web root.

Step 4.1: Get your SHA-256 Signature Fingerprint

Run the following command to retrieve the SHA-256 fingerprint from your keystore:

```
keytool -list -v -keystore android.keystore -alias android -storepass YOUR_PASSW
```

Locate the line starting with `SHA256:`. It will look like this: `9A:2F:5C:88:...:E4:FA:21`

Step 4.2: Create the assetlinks.json file

Create a file named `assetlinks.json` with the following structure. Replace `YOUR_APP_PACKAGE_NAME` (e.g., `in.lawoncall.twa`) and `YOUR_SHA256_FINGERPRINT` with your actual values:

```
[
  {
    "relation": ["delegate_permission/common.handle_all_urls"],
    "target": {
      "namespace": "android_app",
      "package_name": "YOUR_APP_PACKAGE_NAME",
      "sha256_cert_fingerprints": [
        "YOUR_SHA256_FINGERPRINT"
      ]
    }
  }
]
```

Step 4.3: Upload to Web Server

Host the file under the following path on your frontend web server:

`https://lawoncall.in/.well-known/assetlinks.json`

> ⚠️ **IMPORTANT:** The response header for the `assetlinks.json` request MUST have a Content-Type of `application/json`. Ensure the folder is not blocked by custom Nginx rules or AWS S3 cache settings!

5. Publishing on Google Play Console

Once your signed AAB and Digital Asset Links are configured, you are ready to publish:

1. Log in to the [Google Play Console](https://play.google.com/console).
2. Click **Create app** and enter your App Name, Default Language, and App Type (App, not Game).
3. Complete all initial setup tasks under the dashboard:
   - **Privacy Policy:** Add the link to your website's privacy policy.
   - **Content Rating:** Complete the questionnaire to get a rating for your app.
   - **Target Audience:** Select target ages (e.g., 18+ for legal apps).
4. Navigate to **Main store listing** under **Grow users** -> **Store presence**:
   - Upload your app icon (512x512), feature graphic (1024x500), and screenshots.
   - Add Short and Full descriptions of the LawOnCall services.
5. Navigate to **Production** under **Release** -> **Production** and click **Create new release**.
6. Upload the ** `app-release-signed.aab` ** file and submit the release for review!